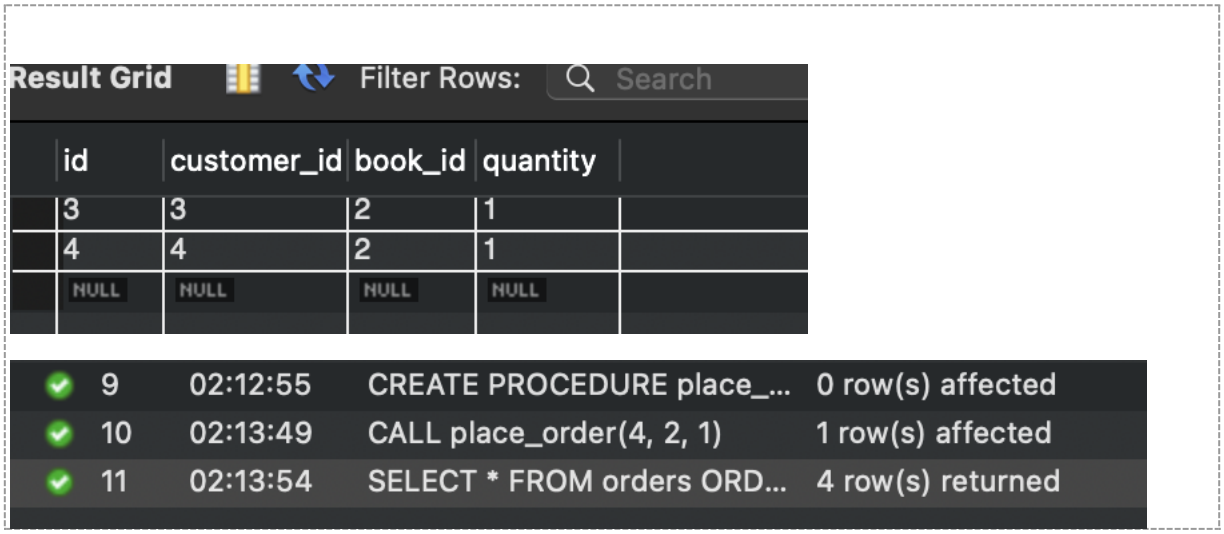
Assignment: Lab 11 (Week 13)

Name: touch-ing yingyong ID: 6722782164

1. Stored procedure

place_order inserts one row into orders. CALL place_order(4, 2, 1) creates order id 4 for customer 4 (book 2, quantity 1).



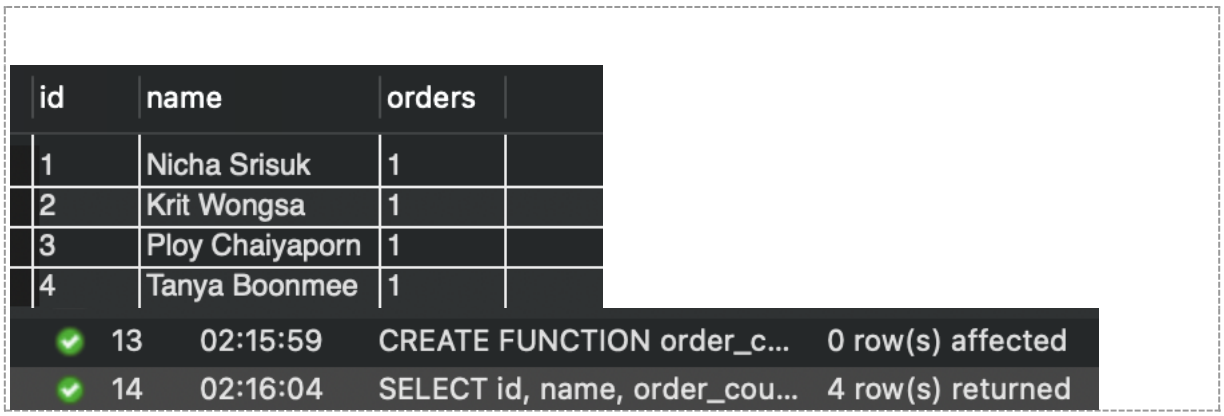
Result Grid Filter Rows: Search

id	customer_id	book_id	quantity
3	3	2	1
4	4	2	1
NULL	NULL	NULL	NULL

9	02:12:55	CREATE PROCEDURE place_...	0 row(s) affected
10	02:13:49	CALL place_order(4, 2, 1)	1 row(s) affected
11	02:13:54	SELECT * FROM orders ORD...	4 row(s) returned

2. Stored function

order_count(p_customer) counts the rows in orders for that customer (READS SQL DATA). The SELECT calls it once per customer.

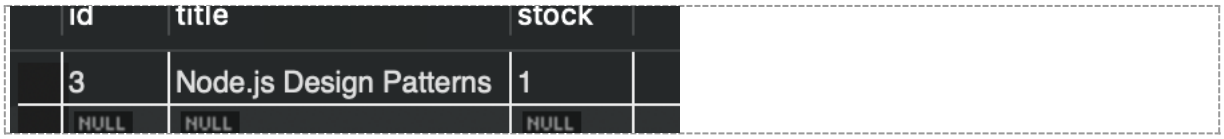


id	name	orders
1	Nicha Srisuk	1
2	Krit Wongsa	1
3	Ploy Chaiyaporn	1
4	Tanya Boonmee	1

13	02:15:59	CREATE FUNCTION order_c...	0 row(s) affected
14	02:16:04	SELECT id, name, order_cou...	4 row(s) returned

3. Trigger with a stock guard

The order for 2 copies of book 3 takes the stock from 3 to 1 with no UPDATE written by me; the trigger did it.



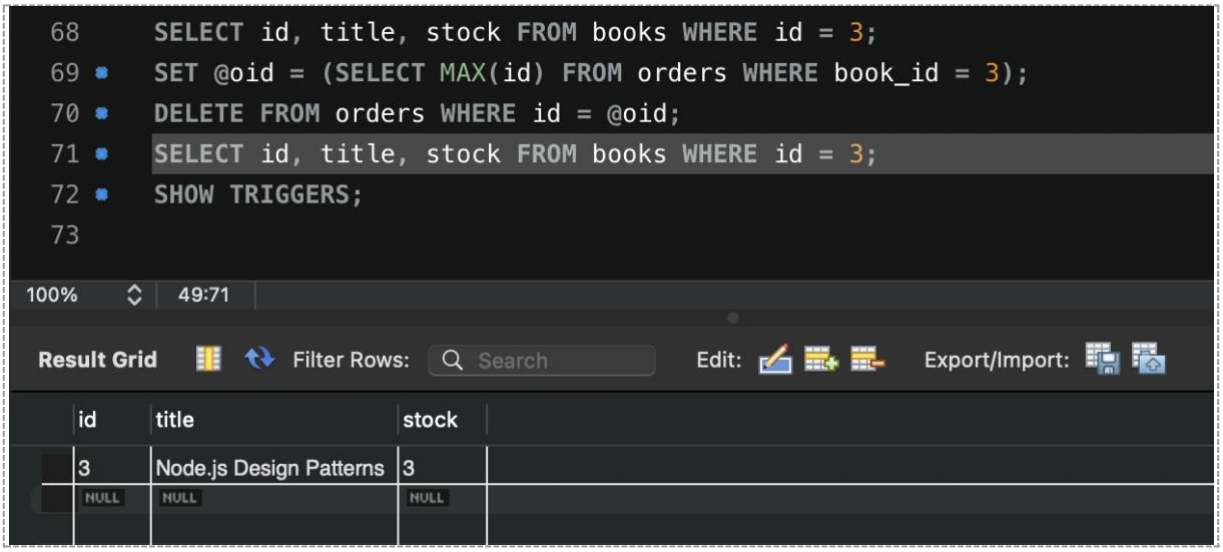
id	title	stock
3	Node.js Design Patterns	1
NULL	NULL	NULL

Ordering 3 copies of book 5 (only 1 in stock) fails with Error 1644 "not enough stock", and no order for book 5 exists.



4. Trigger (reverse)

Deleting the book 3 order fires after_order_delete, which adds OLD.quantity back: the stock goes from 1 back to 3.



5. Use the routines from the web app

POST now makes one call, CALL place_order(?, ?, ?). SQLSTATE 45000 is mapped to 409 and a foreign key error to 404. DELETE keeps only the DELETE; the trigger restores the stock. GET /customers adds order_count(id) AS orders.

```
exports.placeOrder = async (req, res) => {
  const customerId = Number(req.params.id);
  const bookId = Number(req.body.book_id);
  const quantity = Number(req.body.quantity);

  if (!isPosInt(customerId) || !isPosInt(bookId) || !isPosInt(quantity)) {
    return res.status(400).json({
      error: 'customer id, book_id and quantity must be positive integers' });
  }

  try {
    await pool.execute('CALL place_order(?, ?, ?)', [customerId, bookId, quantity]);
    res.status(201).json({ ok: true });
  } catch (err) {
    if (err.sqlState === '45000') {                    // our SIGNAL
      return res.status(409).json({ error: err.sqlMessage });
    }
    if (err.code === 'ER_NO_REFERENCED_ROW_2') {       // FK: no such customer or book
      return res.status(404).json({ error: 'customer or book not found' });
    }
    res.status(500).json({ error: err.message });
  }
};
exports.cancelOrder = async (req, res) => {
  const customerId = Number(req.params.id);
  const orderId = Number(req.params.orderId);
  if (!isPosInt(customerId) || !isPosInt(orderId)) {
    return res.status(400).json({ error: 'ids must be positive integers' });
  }

  try {
    const [r] = await pool.execute(
      'DELETE FROM orders WHERE id = ? AND customer_id = ?', [orderId, customerId]);
    if (r.affectedRows === 0) return res.status(404).json({ error: 'order not found' });
    res.json({ ok: true, order_id: orderId });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

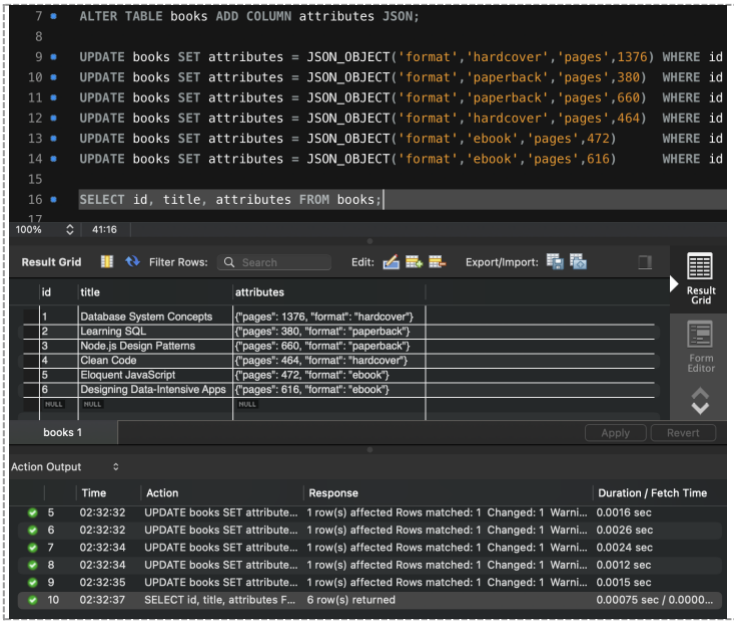
Responses

Clear

```
Book 5 stock: 1 → 1 (0) ✔
  409  POST /customers/2/orders {book_id:5, quantity:3} → {"error":"not enough stock"}
Book 2 stock: 2 → 5 (+3) ✔
  200  DELETE /customers/2/orders/8 → {"ok":true,"order_id":8}
Book 2 stock: 5 → 2 (-3) ✔
  201  POST /customers/2/orders {book_id:2, quantity:3} → {"ok":true}
```

6. JSON column

books gets a JSON column attributes; every book stores {format, pages} built with JSON_OBJECT. Books 5 and 6 are ebooks.



7. Query JSON

attributes->>'$.format' pulls the format out of the JSON as plain text; the second query uses it in WHERE to keep only the ebooks.

title	format
Database System Concepts	hardcover
Learning SQL	paperback
Node.js Design Patterns	paperback
Clean Code	hardcover
Eloquent JavaScript	ebook
Designing Data-Intensive Apps	ebook

id	title	pages
5	Eloquent JavaScript	472
6	Designing Data-Intensive Apps	616

GET /books now returns format and pages, and GET /books?format=ebook adds WHERE attributes->>'$.format' = ? with the format passed as a parameter.

```
exports.list = async (req, res) => {
  try {
    const { format } = req.query;
    const [rows] = await pool.execute(
      `SELECT id, title, author, price, stock,
              attributes->>'$.format' AS format,
              attributes->>'$.pages'  AS pages
         FROM books
        ${format ? "WHERE attributes->>'$.format' = ?" : ''}
        ORDER BY id`,
      format ? [format] : []);
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

Books stock: Q3–Q5 · format & pages: Q6–Q7

Filter by format (a JSON field)

ebook

ID	Title	Author	Price (THB)	Stock	Format	Pages
5	Eloquent JavaScript	Haverbeke	750.00	1	ebook	472
6	Designing Data-Intensive Apps	Kleppmann	1690.00	0 (sold out)	ebook	616

Books come from GET /books (or GET /books?format=ebook). Format and pages are read out of the attributes JSON column with ->>.

8. Short answer

(a) A good use for a trigger is keeping related data in step automatically: here every order reduces the stock and every cancel gives it back, no matter which app or query made the change, so the rule cannot be forgotten. The risk is that triggers are hidden side effects. Before Q5 was fixed, the Node code still updated the stock itself and the trigger did it again, so one order took the quantity off the stock twice, and nothing in the JavaScript showed why.

(b) I would choose a JSON column when the data is flexible or differs from row to row, like format and pages, which may not apply to every product and may get new fields later, and when it is mostly read together with its row. If the field is needed on every row, searched or joined often, or needs constraints and foreign keys, an ordinary column or a new table is better.